WhisperCatcher: Demystifying Unauthorized and Encrypted Private Data Transmission in Android Applications

Zhaoyu Qiu
Xi'an Jiaotong University
Xi'an, China
qiuzhaoyu@stu.xjtu.edu.cn

Ming Fan*
Xi'an Jiaotong University
Xi'an, China
mingfan@mail.xjtu.edu.cn

Bocan Ma
Xi'an Jiaotong University
Xi'an, China
bocanma@stu.xjtu.edu.cn

Yutian Tang
University of Glasgow
Glasgow, United Kingdom
Yutian.Tang@glasgow.ac.uk

Lei Xue
School of Cyber Science and Technology, Sun Yat-sen University
Shenzhen, China
xuelei3@mail.sysu.edu.cn

Haijun Wang
Xi'an Jiaotong University
Xi'an, China
haijunwang@xjtu.edu.cn

Ting Liu
Xi'an Jiaotong University
Xi'an, China
tingliu@mail.xjtu.edu.cn

Abstract

The privacy issues associated with Android apps are increasingly raising our concerns. Unfortunately, a large portion of privacy breaches in Android apps cannot be accurately detected by existing approaches, especially private data that is collected without consent and transmitted in encrypted form. Even if existing studies are able to break the encryption at protocol level to recover the structure and content of traffic packets, they are still unable to understand the code layer encrypted data. To solve this problem, we propose WhisperCatcher, an automated tool for analyzing unauthorized and encrypted private data transmitted by apps. For each app, WhisperCatcher first captures the raw traffic generated during the app's startup phase, before the user consents to the privacy policy, and then extracts the semantic information. Furthermore, it utilizes the traffic semantics to guide static code analysis and extracts transmission-related key functions. Finally, it performs dynamic instrumentation analysis and recovers the encrypted data, thereby identifying unauthorized private data transmissions. Extensive evaluations show that WhisperCatcher significantly outperforms existing tools, and it achieves the recall of 91.38% and F1-Score of 95.49%, respectively. In addition, we conduct a large-scale measurement analysis on 14,879 apps and WhisperCatcher identifies 13,966 traffic flows from 4,966 apps that transmit private data prior to obtaining user consent, among which 3,838 (27.48%) flows contain app-encrypted data. Our findings highlight the potential privacy leakage risks in Android apps, which should be brought to the attention of the community.

*Corresponding author.

This work is licensed under a Creative Commons Attribution 4.0 International License.
ICSE '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2025-3/26/04
https://doi.org/10.1145/3744916.3773154

CCS Concepts

• **Security and privacy → Software security engineering**.

Keywords

Private Data Transmission, Program Analysis, Android Apps

ACM Reference Format:
Zhaoyu Qiu, Ming Fan, Bocan Ma, Yutian Tang, Lei Xue, Haijun Wang, and Ting Liu. 2026. WhisperCatcher: Demystifying Unauthorized and Encrypted Private Data Transmission in Android Applications. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3744916.3773154

1 Introduction

Android apps hold a dominant share of the mobile app market and have become essential parts of our daily lives. As one of the most prevalent communication channels, they play significant roles in private data collection and transmission. However, many apps excessively collect user's private data for user tracking and personalized advertising recommendations, and these privacy issues are increasingly raising concerns [45, 46, 60]. In response to these privacy challenges, a series of laws or regulations has been enacted, such as the General Data Protection Regulation (GDPR), the California Consumer Privacy Act (CCPA), and China's Personal Information Protection Law (PIPL). The aim of introduced laws or regulations is to ensure the transparent processing of private data and protect user's privacy rights.

Motivation. Existing studies have made significant progress in detecting privacy issues of mobile apps, including privacy leaks from third-party libraries [28, 35, 54], leaks caused by permission abuse [13, 66], and the inconsistency of data practices and privacy policies [4, 57, 69]. However, these studies rarely focus on app behaviors that transmit private data without user's authorization [36], especially when the private data is encrypted before transmission

(**Motivation A**). Such unauthorized transmissions frequently occur before the user consents to the privacy policy, clearly violating user's privacy rights and breaching existing regulations. Specifically, the GDPR stipulates that the collection and transmission of user's personal data must have their voluntary and explicit consent. We call this unauthorized data transmission **UDT** in this work.

To detect and understand app's UDT behaviors, network traffic analysis is an effective approach. The traffic flows of apps can be categorized into three types: 1) plaintext traffic flows, in which the data is transmitted without any data protection measures; 2) HTTPS-encrypted traffic flows, in which the data is protected using the TLS/SSL protocols; and 3) app-encrypted traffic flows, in which the data is encrypted or encoded within the app code before the network transmission using universal protocols. Most existing studies are limited to plaintext traffic flows or HTTPS-encrypted traffic flows [17, 30, 44], and they cannot handle app-encrypted traffic flows, which hinders the accurate identification of UDT behaviors. To the best of our knowledge, the most relevant work on analyzing app-encrypted traffic is ThirdEye [41], which instruments official encryption APIs to examine custom-encrypted data. However, it fails to handle encryption implemented via non-official libraries or complex encoding schemes. This highlights the need for more robust approaches to detect and understand UDT behaviors under complex encryption or encoding scenarios (**Motivation B**).

Our Work. In this study, we propose WhisperCatcher, an automated tool for analyzing Android apps' unauthorized private data transmissions. Specifically, WhisperCatcher is based on the following three insights. 1) the app's raw traffic flows contain useful and critical semantics information, such as the domain of URL, path segments of URL and query parameters, even if the transmitted private data is encrypted. 2) the encrypted transmission logic in the app code can be located using critical semantics, which can be found in static string fields of classes, string references in function bodies and so on. 3) the plaintext form of encrypted or encoded data usually appears in app code as arguments or return values of specific functions. Based on these insights, WhisperCatcher first captures the raw traffic flows during the app's startup phase, before the user has consented to the privacy policy and then extracts traffic semantics by identifying keywords from plaintext portions of traffic. Further, it leverages traffic semantics to guide static code analysis, with the goal of obtaining key call chains that contain both traffic semantics and encryption/encoding functions. Finally, it instruments functions within key call chains during dynamic execution to retrieve arguments and return values, so the transmitted data in the plaintext form can be obtained.

To thoroughly evaluate WhisperCatcher, we built a comprehensive dataset of 14,879 apps sourced from AppChina, Google Play, Anzhi, Wandoujia, and APKPure, spanning the period from 2018 to 2025. From this dataset, we randomly selected 248 apps exhibiting unauthorized data transmission behaviors to construct the benchmark dataset, and manually validated their transmission and encryption logic. The evaluation results on the benchmark dataset indicated that, for private data transmitted in plaintext or over HTTPS, similar to most existing tools, WhisperCatcher achieves consistently high recall and F1-Score. But for private data transmitted after app encryption, WhisperCatcher achieves a recall of 91.38% and an F1-Score of 95.49%, representing improvements of 43.25% and 30.50% over current state-of-the-art approach, respectively. We further conducted a large-scale measurement analysis based on the comprehensive dataset. A total of 13,966 traffic flows from 4,966 apps were identified as transmitting private data prior to obtaining user consent, among which 3,838 (27.48%) contained app-encrypted data.

In summary, this work makes the following contributions.

- We introduce WhisperCatcher, an automated and effective tool for identifying unauthorized and encrypted data transmissions. WhisperCatcher employs traffic-semantics-guided static code analysis combined instrumentation to recover encrypted data, making it more adaptable to different app implementations and capable of accurately identifying transmitted data.
- The comprehensive evaluations show that WhisperCatcher achieves a recall of 91.38% and an F1-Score of 95.49%, outperforming the state-of-the-art approach by 43.25%, 30.50%, respectively.
- We conduct a large-scale measurement analysis on 14,879 real-world apps and identify 13,966 traffic flows from 4,966 apps that transmit private data before user consents to the privacy policy, among which 3,838 (27.48%) contain app-encrypted private data.
- To foster reproducibility and facilitate further studies, we have open-sourced WhisperCatcher, with the implementation available at https://github.com/HARP-I/whispercatcher.

2 Background

2.1 Data Transmission Paradigm in Apps

In mobile apps, private data typically undergoes several stages, from being collected on the user's device to being transmitted to the server, including data packaging, encoding, encryption, and transmission, as shown in Table 1.

2.1.1 Data Packaging. The data packaging stage is the most important part in the entire process, which is responsible for collecting user's private data and organizing it into an appropriate format. Collected data is usually organized in the form of key-value pairs, e.g., the HashMap is a common structure where each entry contains the privacy data type and the corresponding value. During the actual transmission, the HashMap data is further transformed into a JSON structure for subsequent processing. Another common organization format is the URL format, where the keys and values of private data are concatenated through "=", and different types of data are concatenated through "&". Finally, the collected data forms a long string. When data is packaged, an app may transmit it directly using a secure protocol like HTTPS, while others may encode and encrypt it before transmission.

2.1.2 Encoding. After data collection, the app may perform encoding on the data to reduce processing and transmission overhead. Encoding may be applied to the encrypted ciphertext or directly to the original plaintext data. During the encoding process, the app may use the standard encoding scheme, such as Base64 encoding and URL encoding. In addition, customized encoding schemes can also be adopted to satisfy particular requirements. Notably, in some cases the data is transmitted directly after encoding, whereas in others, the encoded data also needs to be encrypted prior to transmission.

Table 1: Private data transmission paradigm.

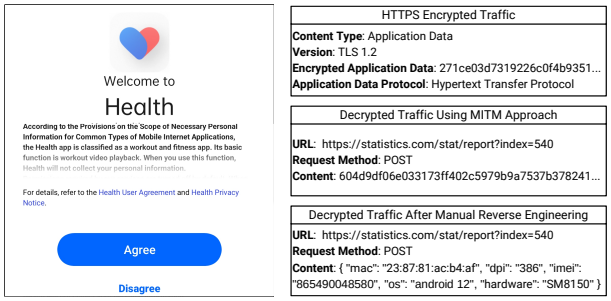
Stage	Type	Example Code

Data Packaging	Key-value Pairs	`Map<String, String> data = new HashMap<>();`
		`data.put("mac", getMacAddress()); // map entries`
		`String data = "platform=android&os=12"; // URL format`

Encoding	Standard Encoding Customized Encoding	`String encoded = Base64.encodeToString(data);` `String customEncoded = EncodeUtils.encode(data);`

Encryption	Official Library Encryption
	Third-Party Library Encryption
	Customized Encryption
		`Cipher cipher = Cipher.getInstance("AES/CBC/PKCS7Padding");`
		`Aead aead = keysetHandle.getPrimitive(Aead.class);`
		`String customEncrypted = CryptoUtils.encrypt(data);`

Transmission	Official Library Transmission Third-Party Library Transmission	`HttpURLConnection conn = (HttpURLConnection) url.openConnection();` `Request req = new Request.Builder().url(url).post(body).build();`



(a) Startup Activity (b) App traffic under different analysis approaches

Figure 1: Unauthorized and encrypted data transmission.

2.1.3 Encryption. Before data transmission, the app may choose to encrypt the collected data or encoded data to ensure security. These encryption algorithms typically come from official encryption libraries (e.g., APIs in the `javax.crypto` and `java.security` packages), third-party encryption libraries (e.g., Google Tink Library [27]) and developer-customized encryption algorithms [61]. It should be clarified that the encryption mentioned here comes from the app code level, rather than standard protocols, which means that the data may be protected by both app-level encryption and protocol-level encryption.

2.1.4 Transmission. Finally, the processed data is transmitted to the server, with the communication protocol and configuration determined at this stage. The implementation of communication functions typically relies on official libraries (e.g., APIs in the `java.net` package) or third-party libraries (e.g., OkHttp Library [51]).

2.2 Unauthorized Data Transmission

As one of the most frequently used devices in our daily lives, mobile phones contain a wide range of private information about us, which many apps seek to access for user tracking, targeted advertising, and personalized recommendations. To protect user's privacy rights, a series of regulations have been enacted. For example, GDPR states that *data processing shall be lawful if the data subject has given consent to the processing of his or her personal data for one or more specific purposes.* in Art.6 [25]. Concurrently, China's PIPL also highlights the significance of user's consent, it states that *personal information processors need to obtain individual consents before processing personal information* in Art.13 [40].

Although strict privacy compliance requirements have been imposed by app markets, some apps still collect and transmit user's private data without obtaining explicit authorization. We present a data transmission case in Figure 1a, in the initial launch of target app, the user encounters a privacy policy requesting consent. However, before the user clicks the "Agree" button for authorization, the app has already started collecting and transmitting user's private data in the background. Undoubtedly, such unauthorized data transmission violates compliance requirements and significantly undermines user's privacy rights.

```
public class Statistics {
    private static final String base = "https://statistics.com/stat/report";
    // ... other fields
    public void commWithServer() {
        Map<String, String> privateData = collectPrivateData();
        post(privateData);
    }
    private Map<String, String> collectPrivateData() {
        Map<String, String> data = new HashMap<>();
        data.put("mac", getMacAddress());
        data.put("dpi", getDpi()); // data packaging
        // ... other private data
        return data;
    }
    private void post(Map<String, String> data) {
        String url = base + "/?index=" + getIndex();
        HttpPost httpPost = new HttpPost(url);
        init(httpPost);
        String jsonData = toJson(data); // formatting
        // customized encryption or encryption using non-official libraries
        byte[] bytes = EncryptUtils.encrypt(jsonData);
        httpPost.setData(bytes);
        execute(httpPost); // transmission
    }
    // ... other methods
}
```

Listing 1: A simplified example of encrypted private data transmission

2.3 Motivating Example

To recognize and understand the unauthorized private data transmission by apps, the traffic flow is a promising perspective. However, the data within the traffic may be encrypted in different ways, including protocol-level encryption, app-encryption, or a combination of both. There are several studies focusing on encrypted traffic

flows analysis of apps, including [16, 17, 36, 41]. These approaches can be categorized into MITM(man-in-the-middle)-based analysis and instrumentation-based analysis. The MITM-based analysis involves configuring the app to accept a forged certificate from a proxy server, allowing the interception of communications. This enables real-time decryption at HTTPS layer and facilitates analysis of transmitted content. The instrumentation-based analysis directly instruments the app's functions, enabling the extraction of plaintext data before it is encrypted and transmitted.

Nevertheless, MITM-based approaches fail to identify data encrypted within the app code. Listing 1 presents the code snippets where private data is encrypted in the app and then transmitted over HTTPS protocol. The transmitted traffic is captured and Figure 1b illustrates the original HTTPS-encrypted traffic, the decrypted traffic using MITM-based approach and the decrypted traffic after manual reverse engineering. It is evident that we cannot infer the transmitted private data from the original HTTPS-encrypted traffic. Moreover, since the data is encrypted within the app code before transmission, the MITM-based approach also fails. Instrumentation-based approaches hold promise for identifying app-encrypted data. To the best of our knowledge, the state-of-the-art instrumentation-based analysis approach is ThirdEye [41], which instruments encryption APIs in official libraries (e.g., `javax.crypto.Cipher.doFinal` and `javax.crypto.Cipher.update`). However, if the app uses third-party libraries or customized encryption (as we have shown in line 21 of Listing 1), or the private data is encoded before encryption, this approach does not work. This necessitates a more comprehensive and robust tool for analyzing unauthorized and encrypted private data transmission in apps. To achieve this goal, there are two main challenges have to be addressed.

Challenge A: How to locate encrypted data transmission related code snippets from the app's massive amount of code. Relying on fixed encryption APIs is insufficient to accommodate the diverse encryption methods adopted by different apps. Therefore, it is necessary to extract code snippets related to encrypted data transmission from the entire app for further analysis. However, the app code typically contains a large amount of functional logic beyond data transmission, making it highly challenging to accurately identify and extract these code snippets from the massive codebase.

Challenge B: How to accurately identify private data under transmission. The data transmitted by apps exhibits complex forms. For instance, the names of some key-value pairs packaged data are abbreviated or meaningless (e.g., "mc" and "mac" both represent MAC addresses), while the same type of data may have different representations (e.g., both "386*383" and "386.66" can be used as "dpi" values). Traditional rule-based approaches fail to comprehensively cover the diverse representations of private data, making it challenging to accurately identify the types of private data transmitted by apps.

3 Design of WhisperCatcher

In this section, we provide a detailed design of WhisperCatcher. As shown in Figure 2, WhisperCatcher comprises four main distinct phases, including raw traffic capturing, traffic semantics analysis, transmission functions identification as well as private data extraction and analysis. For captured plaintext traffic, WhisperCatcher directly extracts the transmitted private data for UDT detection. For captured encrypted traffic, WhisperCatcher performs traffic-semantics-guided static code analysis to identify functions related to transmission and encryption. It then instruments these key functions to extract the actually transmitted private data, enabling effective UDT detection.

3.1 Raw Traffic Capturing

The initial step is to capture the target app's traffic. Here we use MITM-based traffic capturing approach since it is relatively mature and has been widely adopted in existing studies [36, 41]. The captured traffic flows may contain unrelated system traffic, and these traffic flows are filtered out to avoid interference. Finally, the captured traffic flows are categorized into plaintext traffic and encrypted traffic for further analysis.

3.1.1 MITM-based Traffic Capturing. MITM-based approaches are able to intercept app traffic and decrypt HTTPS-encrypted content in real time. In addition, they are also capable of capturing plaintext traffic transmitted over HTTP. This meets our requirements and therefore adopted for traffic capturing. Furthermore, DroidSSLUnpinning [21] is leveraged to detect and disable SSL certificate pinning. With this setup, we are able to capture most apps' traffic.

During the traffic capturing phase, we grant required permissions to the target app, launch it, and monitor its data transmission for one minute. In this process, we do not interact with target app (e.g., agree to privacy policy), it ensures that the captured traffic solely reflects the default behavior of the app, and unaffected by user's operations [36]. After traffic capturing, the URL, request method, request header, and content of transmitted traffic are recorded. It is worth noting that if these traffic semantics are available, other traffic capturing approaches can also be adopted, and MITM-based approach is not limited.

3.1.2 Unrelated Traffic Filtering. Before analyzing the captured traffic, it is necessary to filter out unrelated flows, as traffic from the system or other apps may contaminate that of the target app. So we implement a traffic blacklist to exclude these unrelated traffic flows. We maintain the test device in a silent state for more than 24 hours, ensuring that only system apps are present. During this period, we capture the generated traffic flows and record the corresponding domains to build a blacklist. During the testing, we keep the target app as the only user-installed app in the environment, and then apply the blacklist to filter out unrelated traffic flows.

3.2 Traffic Semantics Analysis

In order to locate the code of encrypted private data transmission, we need to extract semantics from raw traffic. The semantics extraction is based on the following observation: the bodies of private data transmission functions may contain keywords from the raw traffic. For example, the domain name of URL, the path segments of URL, custom fields in the request headers, etc. To this end, we propose the following approach for traffic semantics extraction.

- The request URL is divided into request domain and path segments. These components can help to find URL constructing functions.

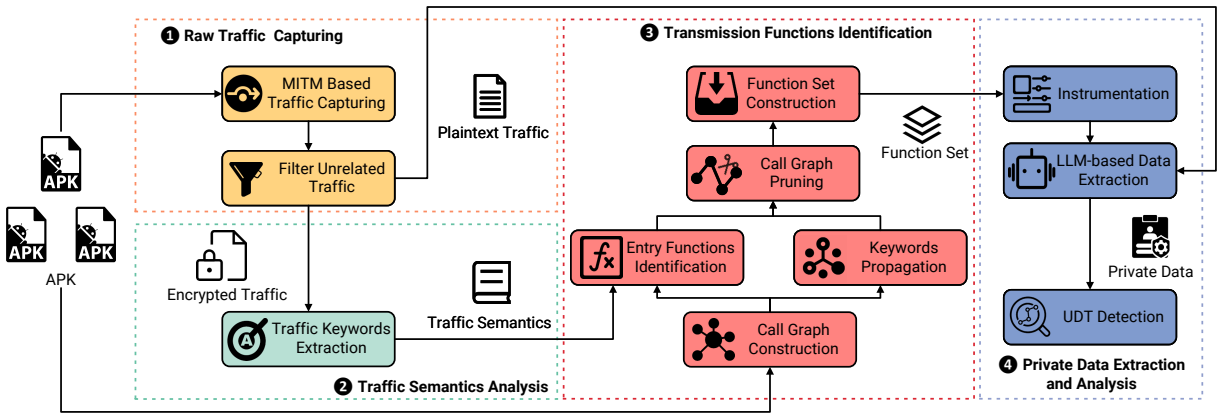


Figure 2: Overview of WhisperCatcher.

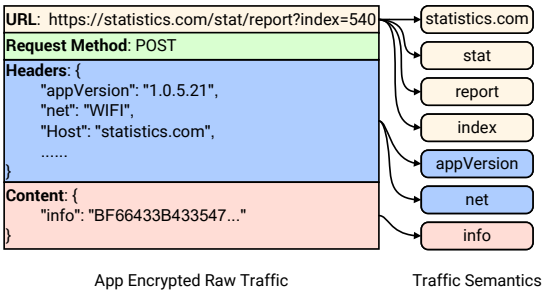


Figure 3: A traffic semantics extraction example.

- If the request URL contains query parameters, these parameter keys are extracted, as they may indicate data fields within the data transmission process.
- Custom fields in the request header are extracted, since app may use custom header fields to mark special traffic flows, these special fields can help to locate specific data transmission code.
- For request bodies that can be interpreted as JSON format, the parameter keys will be extracted to assist in locating data packaging functions within the app.
- Other complex information within the traffic content is processed using regular expressions, and we seek to extract "keys" from traffic content. For example, we use the regular expression to extract `encData` from the traffic content of `encData={"1": "CBF32BE5B0..."}`.

We illustrate a traffic semantics extraction example in Figure 3, the left part presents one app-encrypted raw traffic and the value of "info" field in traffic content is encrypted, besides, the right part presents extracted traffic semantics. According to our approach, the URL is divided into domain and path segments. In addition, the query parameter "index" is also extracted. Further, since there are custom header fields in the raw traffic, these fields are extracted, i.e., "appVersion" and "net". Finally, since the traffic content can be interpreted as JSON format, the parameter key is extracted, i.e., "info".

For each encrypted traffic instance, we perform the same semantics extraction process, and generate a corresponding set of keywords for subsequent analysis.

3.3 Transmission Functions Identification

In this phase, we leverage traffic semantics to accurately locate functions related to encryption and transmission from the app code, thus we can identify what private data is encrypted and transmitted. Figure 4 presents the workflow of our approach.

3.3.1 Call Graph Construction. First, we construct the call graph of the target app based on soot [49] and FlowDroid [8]. In our constructed graph, each node represents a function, and each edge signifies a function call, which connects the caller function and the callee function. During this process, all function bodies of those function nodes are recorded for further analysis.

3.3.2 Entry Functions Identification. Given the traffic semantics and target app's call graph, we identify functions that contain semantic keywords as entry functions. We focus on strings in the function bodies, including the strings from regular assignment statements, the string arguments of invoke expressions, and the strings from class field assignment statements. It should be noted that, if we find strings from class field assignment statements (e.g., from class initializing functions), these fields and classes will be recorded.

Functions containing the same string are grouped together. If this string fully or partially matches any of the traffic semantic keywords, then all such functions are considered as entry functions, so we can effectively retrieve those entries.

3.3.3 Keywords Propagation. After the identification of entry functions, we find that those class initializing functions cannot be easily used to guide the process of locating encryption and transmission functions, because these initialization functions may only assign values to class static fields, and there is no need for other functions to explicitly call them (as we have shown in line 2 of Listing 1). However, these initializing functions may contain significant traffic semantics, for example, the URL of some traffic flows. In order to

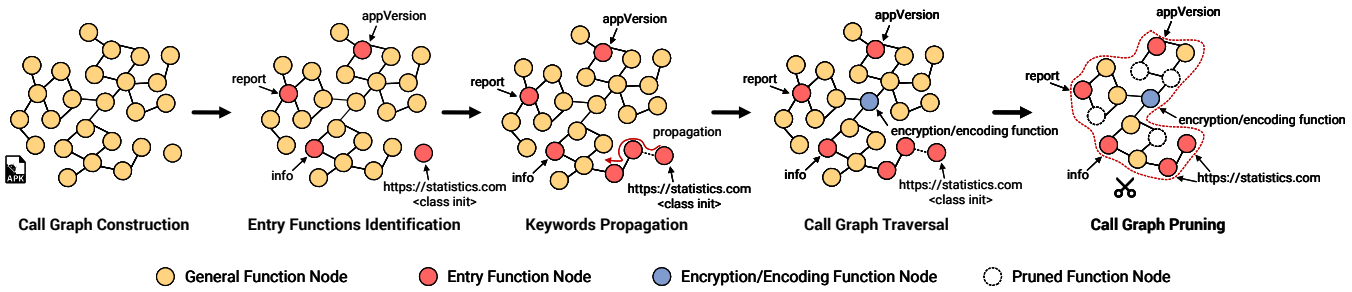


Figure 4: Workflow of the transmission functions identification.

Algorithm 1: Graph traversal

Input: the entry set, *Entries*
Output: the function set from key call chains, *Functions*

```
callchains ← ∅ ;                // the set of callchains
for entry_i ∈ Entries do
    DFT(entry_i, [], callchains);
Functions ← getFunctionSetFrom(callchains);
return Functions;
Function DFT(node, path, callchains):
    path.append(node) ;         // the recorded node path
    if node is encryption/encoding function then
        callchains ← callchains ∪ {callchainFrom(path)};
    else
        for child ∈ node.children do
            if child ∉ path then
                DFT(child, path, callchains);
    path.pop();
```

take these semantics into consideration, we propose the keywords propagation mechanism.

In the entry functions identification process, we have recorded the string fields and classes in the initializing functions, so we search for functions within the corresponding class that contain these fields. If the values of recorded fields fully or partially match the traffic semantic keywords extracted from Section 3.2, we also consider those functions that reference them as entry functions. In addition, these functions are further propagated to their callers to mitigate the impact of multi-layer nested calls on key functions' identification.

In practice, we empirically limit the propagation depth to four layers to balance effectiveness and time consumption, and this parameter can be flexibly configured according to requirements.

3.3.4 Graph Traversal and Pruning. Given the entry functions, we employ depth-first traversal algorithm to explore the entire function call graph, aiming to identify significant call chains from entry functions to encryption/encoding functions. Given the truth that encryption/encoding functions are rarely being obfuscated, we can utilize function signature patterns to locate them. If a function signature contains encryption/encoding keywords, such as `"encrypt"`, `"encode"`, `"doFinal"`, `"aes"` or `"rsa"`, it will be regarded as one encryption/encoding function.

It should be noted that not all identified call chains from entry functions to encryption functions are related to UDT during target app's startup phase. In fact, a significant portion of those call chains are not fully executed at launch time. However, due to extensive code reuse in apps, these call chains are likely to contain reused data packing and encoding functions, and these functions can be instrumented to retrieve private data in plaintext form.

Based on this insight, we extract all call chains from entry functions to encryption/encoding functions, and all function nodes on these chains are retained, while other irrelevant functions are ignored. The overall algorithm is presented in Algorithm 1. These call chains incorporate both traffic semantics and encryption/encoding contexts, which enhances WhisperCatcher's capability to diverse implementation scenarios. As a result, regardless of whether an app employs encryption APIs from official libraries, utilizes third-party implementations or adopts complex encoding schemes, WhisperCatcher has the potential to recover plaintext data.

Even though we obtain these functions from key call chains, not all of them are worth instrumenting. For example, functions without parameters and with a return type of `void` are generally useless, as no private data can be extracted from their arguments or return values. To characterize the worth of each function, we conducted manual reverse engineering and summarized the common features of functions that truly handle private data. Specifically, a function is considered privacy-related if its parameters or return value are of certain data types, including `java.lang.String`, `java.util.Map`, `java.util.List`, `byte[]`, `org.json.JSONObject`, or `java.io.File`. The privacy-related functions are therefore extracted to form the key function set for instrumentation, while others are pruned. This pruning process effectively mitigates the influence of irrelevant functions and reduces the resource overhead during instrumentation.

3.4 Private Data Extraction and Analysis

Given the set of functions related to private data encryption and transmission, the next step is to recover the private data in plaintext form and analyze target app's UDT behaviors.

3.4.1 Instrumentation. Based on the observation that private data in plaintext form typically appears in function arguments or return values, we adopt the instrumentation-based approach to recover

Table 2: Privacy data types.

Data Type	Instance
Device Specific	IMEI, IMSI, UDID, DeviceID, Device Model, CPU Model, OemName, OS Version, SIM Serial, Resolution, SD Card Serial, Storage Info
Communication	Operator, Network Type, MAC Address, IP Address, SSID, BSSID, Bluetooth Info
Location& Sensors	Location, Misc.sensors
Personalization Specific	OAID, UUID, AndroidID, AdvertisingID, App List
User Assets	Phone Number, SMS, Account Info, Clipboard Data

Role

You are an expert in mobile application data security.

Task

Your task is to analyze the private data instances from given **{{app runtime record}}**. The record will be provided in JSON format, where *traffic_info* represents target app's `plaintext` traffic record at runtime, *instrumentation_result* represents target app's function execution record at runtime. Please determine if the given record contains any of the data instances listed in the **{{privacy data types table}}**. Output the results in JSON format with *privacy_instances* as key and an array of privacy instances as value, do not output extra text.

Chain-of-Thought

First, conduct a thorough review of all privacy data types and instances listed in given table to ensure a comprehensive understanding. Second, use only the specified privacy instances refrain from creating new ones. Third, infer privacy instances based on contextual features across different scenarios:

- For data with distinctive features such as IP addresses and MAC addresses, as well as tracking identifiers with unique features, please infer them directly.
  Q: {"traffic_info": {"addr": "192.168.137.238"}}
  Reasoning: This record is a plaintext traffic record, and it contains a key "addr", which does not match any predefined privacy instances in the table. However, the value "192.168.137.238" conforms to the IP address pattern and is also listed in the provided table. Therefore, this record contains an IP Address instance.
  A: {"privacy_instances": ["IP Address"]}
- In cases where the key or value cannot be definitively inferred, the presence of either one can be considered indicative of a privacy instance.
  Q: {"traffic_info": {"": "android 12"}}
  Reasoning: This record is a plaintext traffic record and contains a key of an empty string, making its meaning unclear. However, the corresponding value is "android 12", which can be interpreted as an OS Version. Since OS Version is listed in the provided table, this record is determined to contain OS Version instance.
  A: {"privacy_instances": ["OS Version"]}
- When privacy instance appears in abbreviated or obfuscated forms, its original meaning should be inferred based on contextual information or value patterns.
  Q: {"instrumentation_result": {"b.c.enc": {"args": ["{\"device.sz\": \"1080x2340\"}"], "ret": "E20BA7D32..."}}
  Reasoning: This record is a function execution record. The executed function is "b.c.enc", and its argument is ["{\"device.sz\": \"1080x2340\"}"]. The return value is "E20BA7D32...", which appears to be meaningless. The argument contains a JSON-formatted string in which the key "device.sz" is likely an abbreviation of "device.size", and its value is "1080x2340", which can also be interpreted as a screen resolution. Since the screen resolution is listed as Resolution in the provided table, based on the contextual information, this record contains a Resolution instance.
  A: {"privacy_instances": ["Resolution"]}

Finally, collecting all detected privacy instances in an array, then format the output as a JSON object with the key *privacy_instances*.

Example

Example1
Input: {"traffic_info": {"device.ov": "12", "mc": "00:01:6C:06:A7:29"}}
Output: {"privacy_instances": ["OS Version", "MAC Address"]}

Example2
Input: {"instrumentation_result": {
"com.data.AES128Encode": {
"args": ["{\"md\": \"Redmi_M2007J22C\", \"conn_type\": \"wifi\"}"],
"ret": "9b7b257581..." }}}
Output: {"privacy_instances": ["Device Model", "Network Type"]}

Figure 5: Prompt template for private data extraction.

the actual transmitted data. Specifically, we hook every function in the key function set. During the reinstallation and execution of the target app (under the same conditions of raw traffic capturing, i.e., grant permissions and without interactions), if corresponding functions are called, we are able to capture their arguments and return values, thereby extracting the collected private data without user's authorization.

3.4.2 LLM-based Data Extraction. Extracting private data from function arguments and return values is also challenging. First, the names of some key-value pairs packaged data are abbreviated or meaningless, while the same type of private data may have different representations, so traditional rule-based matching approach is not applicable. Second, the arguments and return values of some functions may contain redundant information, and filtering is necessary. Therefore, we propose a large language model-based (LLM-based) approach for private data extraction.

Based on Android developers' documents [26], existing study [32, 33], and manual analysis, we categorize the frequently encountered private data into the types listed in Table 2. And it will be employed to guide the large language model in private data extraction.

For the plaintext data in traffic flows, we extract it directly for analysis, and for the encrypted data, we extract the instrumentation result of relevant functions for analysis. To guide the large language model in accurately extracting privacy instances from traffic and instrumentation records, we design the prompt template shown in Figure 5. The prompt clearly defines the model's role, the task to be performed, and the expected input-output format. In addition, we incorporate in-context prompting techniques such as chain-of-thought [59] and few-shot prompting [11] to enhance the model's understanding of the extraction process and improve the accuracy of privacy instance identification. If any transmitted private data is found, we determine that the app has UDT behaviors.

4 Evaluation

In this section, we investigate the following research questions (RQs) to evaluate the performance of WhisperCatcher.

- **RQ1: How effective is WhisperCatcher in detecting unauthorized and encrypted private data transmission behaviors compared with existing approaches?**
- **RQ2: To what extent do the two components, namely transmission functions identification and private data extraction, affect the overall performance?**
- **RQ3: How prevalent are UDT behaviors in large-scale real-world apps? What are their characteristics?**

4.1 Implementation

WhisperCatcher is implemented in Python, Java, and JavaScript. We use adbutils [39] for automated device control and testing. In addition, mitmproxy [34] is utilized to capture raw traffic via MITM approach. The static code analysis is powered by soot [49] and FlowDroid [8], and the instrumentation is implemented based on Frida [23]. We utilize python-magic [1] to identify and infer binary data, and employ Gemini 2.5-flash [18] model for private data extraction. Moreover, the evaluation is conducted on a PC equipped with 11th Gen Intel(R) Core(TM) i7-11700F CPU and 32GB RAM, as well as 3 Redmi Note 9 devices with Android 12 (API level 31).

4.2 Dataset

4.2.1 A Large-Scale Dataset. To construct a representative dataset, we used Androzoo [2, 3] to collect Android apps from different app markets, including AppChina, Google Play, and Anzhi. Additionally, in order to expand the sources of Android apps, we also collected apps from Wandoujia [55] and APKPure [6]. We prioritized apps

Table 3: Data transmission detection capabilities of different approaches.

Approach	Plaintext	HTTPS-Enc	App-Enc
Nguyen et al. [36]	✓	✓	✗
TraceDroid [17]	✓	✓	✗
ThirdEye [41]	✓	✓	▲
WhisperCatcher	✓	✓	✓

✓: support, ✗: don't support, ▲: partly support

released in recent years to better reflect the current mobile ecosystem. Finally, we built a large-scale dataset of 14,879 apps, with the period from 2018 to 2025.

4.2.2 Benchmark Dataset. In order to evaluate WhisperCatcher's performance, we also have to construct a benchmark dataset. To the best of our knowledge, there is currently no publicly available dataset, especially a dataset for the study of unauthorized and encrypted private data transmission, so we constructed the benchmark dataset manually. Three postgraduate students with experience in program analysis were involved in constructing dataset, and over 1,500 apps from the large-scale dataset were randomly selected and inspected. We first divided the selected apps into three distinct sets, one for each annotator's initial analysis. Then, these annotators utilized Jadx [47] for decompilation and static analysis, mitmproxy [34] for traffic monitoring, and Frida [23] for instrumentation to identify UDT behaviors. Finally, each annotator's work was cross-validated by the other two. An app was included in the benchmark dataset only if all three annotators confirmed its UDT behavior.

For plaintext and HTTPS-encrypted traffic, 137 apps and 176 traffic flows with obvious private data transmission were finally selected. For app-encrypted traffic, we finally selected 111 apps with unauthorized and encrypted private data transmission behaviors. We spent over 744 hours (about 6.7 hours per app) manually annotating apps' encrypted traffic flows, and 146 encrypted traffic flows were summarized, each associated with the annotated functions that could be used through instrumentation to recover encrypted data. These 248 apps with 322 traffic flows were used for evaluation.

4.3 RQ1: Effectiveness of WhisperCatcher

4.3.1 Baselines. We investigate existing studies [17, 36, 41] focusing on Android apps' private data transmission, and compare their detection capabilities with WhisperCatcher in Table 3. The table illustrates that most approaches cannot handle app-encrypted traffic and only the state-of-the-art approach ThirdEye and WhisperCatcher can handle it. Therefore, for plaintext traffic flows and HTTPS-encrypted traffic flows, we compare WhisperCatcher with all other approaches, while for app-encrypted traffic, we choose to compare WhisperCatcher with ThirdEye.

4.3.2 Effectiveness comparison. We employ the benchmark dataset to evaluate different approaches. For plaintext and HTTPS-encrypted traffic, we construct the ground truth of private data from captured traffic and evaluate whether these approaches can accurately identify it. For app-encrypted traffic, where we have manually annotated the encrypted traffic flows along with their corresponding plaintext form representations, we assess whether both ThirdEye and WhisperCatcher are able to correctly identify the private data. In terms of metrics, we utilize precision, recall and F1-Score to measure the effectiveness of different approaches.

The overall results are presented in Table 4, which indicates that all approaches except TraceDroid achieve high precision, recall and F1-Score in identifying private data from plaintext traffic and HTTPS-encrypted traffic. However, for the task of identifying transmitted private data from app-encrypted traffic, WhisperCatcher achieves the recall of 91.38% and F1-Score of 95.49%, which exceeds

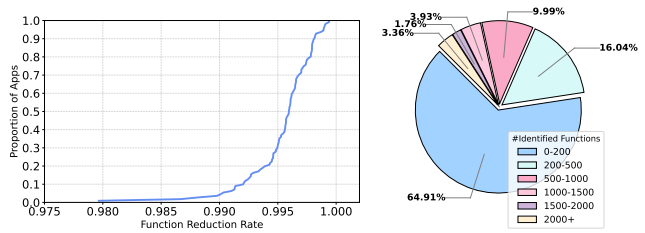
Table 4: Effectiveness comparison of different approaches.

Approach	Traffic Type	Precision	Recall	F1
Nguyen et al. [36]	Plaintext	100%	93.10%	96.43%
	HTTPS-Enc	100%	92.81%	96.27%
	App-Enc	/	/	/
TraceDroid [17]	Plaintext	100%	51.72%	68.18%
	HTTPS-Enc	100%	67.56%	80.64%
	App-Enc	/	/	/
ThirdEye [41]	Plaintext	100%	93.10%	96.43%
	HTTPS-Enc	100%	91.14%	95.36%
	App-Enc	100%	48.13%	64.99%
WhisperCatcher	Plaintext	100%	93.10%	96.43%
	HTTPS-Enc	100%	93.31%	96.54%
	App-Enc	100%	**91.38%**	**95.49%**

Table 5: Detailed results of app-encrypted traffic evaluation on private data instances.

Data Type	Instance	GT.†	ThirdEye		Ours.	
			TP	FN	TP	FN
Device Specific	IMEI	11	7	4	10	1
	IMSI	5	3	2	5	0
	UDID	6	1	5	5	1
	DeviceID	31	8	23	30	1
	OemName	76	46	30	70	6
	Device Model	88	48	40	78	10
	CPU Model	20	4	16	19	1
	OS Version	85	43	42	75	10
	SIM Serial	19	3	16	19	0
	SD Card Serial	2	0	2	1	1
	Resolution	47	17	30	43	4
	Storage Info	15	6	9	14	1
Communication	Operator	9	3	6	8	1
	Network Type	70	38	32	63	7
	MAC Address	31	13	18	27	4
	IP Address	13	5	8	12	1
	SSID	17	6	11	15	2
	BSSID	7	1	6	6	1
	Bluetooth Info	5	1	4	4	1
Location & Sensors	Location	10	6	4	10	0
	Misc.sensors	10	1	9	9	1
Personalization Specific	OAID	7	2	5	7	0
	UUID	68	37	31	64	4
	AndroidID	28	16	12	25	3
	AdvertisingID	45	26	19	43	2
	App List	8	7	1	7	1
User Assets	Account Info	37	25	12	35	2
	Clipboard Data	7	1	6	6	1
Total Instances		777	374	403	710	67

GT.†: Ground truth.



(a) CDF curve of function reduction rate (b) Distribution of identified function counts

Figure 6: Characteristics of identified functions.

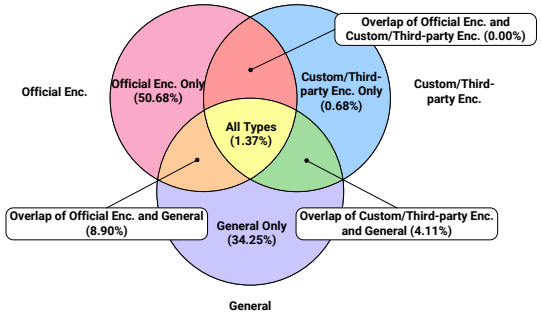


Figure 7: Types of private data extracting functions

ThirdEye by 43.25% and 30.50%, respectively. Since the analysis results all come from app's actual execution, all these approaches have no false positives. The failed cases of WhisperCatcher are further analyzed, 8.62% of the privacy instances fail to be recovered and extracted, among them, 7.21% are caused by app crashes or the failure of instrumentation, which may be attributed to anti-instrumentation measures implemented by apps. Only 1.42% of the failures are due to the inability to locate transmission-related functions. It reflects the power of WhisperCatcher in detecting unauthorized and encrypted private data transmission behaviors.

We present detailed results of app-encrypted traffic evaluation on private data instances in Table 5. We observe that the benchmark dataset covers nearly all categories of private data instances, with the exception of Phone Number and SMS, which are rarely transmitted. In these app-encrypted traffic flows, WhisperCatcher identifies more private data instances than ThirdEye, because WhisperCatcher adaptively sets the instrumented functions based on traffic semantics instead of relying on fixed encryption functions. Besides, WhisperCatcher is also capable of recovering specially encoded private data before encryption based on the instrumentation of call chains, whereas ThirdEye can only extract meaningless encoded data from encryption functions.

Answer to RQ1. The comprehensive evaluations demonstrate that WhisperCatcher is effective in detecting unauthorized and encrypted private data transmission behaviors, achieving the recall of 91.38% and F1-Score of 95.49%, which surpass the current state-of-the-art approach by 43.25% and 30.50%, respectively.

4.4 RQ2: Contribution of Main Components

In this section, we explore the effectiveness of transmission functions identification and the performance of various large language models on private data extraction to answer RQ2.

4.4.1 Transmission Functions Identification. Typical apps comprise tens of thousands to hundreds of thousands of functions, so we first investigate how many data transmission related functions during the startup phase are identified. We analyze the function reduction rate of apps in the large-scale dataset, and the result is presented in Figure 6a. For more than 90% of apps, WhisperCatcher achieves the function reduction rate of over 99%, meaning that even for apps with 300,000 functions, the identified functions remain below 3,000.

In addition, we further analyze the number of these identified functions, as shown in Figure 6b. Most apps have between 0 and 200 identified functions, accounting for 64.91% of all apps. This is followed by apps with 200-500 functions, representing 16.04%. Fewer than 10% of apps contain more than 1,000 identified functions. This indicates that the number of functions requiring dynamic instrumentation is generally limited. This greatly reduces the difficulty of extracting private data from functions' arguments and return values, and also demonstrates WhisperCatcher's strong ability in locating transmission-related functions.

Moreover, we explore the types of functions that can be used for plaintext private data extraction on the benchmark dataset, as presented in Figure 7. We categorize these functions into official encryption/encoding functions, custom/third-party encryption/encoding functions and general functions. The results indicate that only 65.75% of the plaintext data can be retrieved from encryption/encoding functions (both official or custom/third-party), and the remaining 34.25% has to be extracted from regular (non-encryption) functions. This is likely because these apps' encryption/encoding functions are either heavily obfuscated, making them difficult to identify, or implemented at native layer beyond the scope of static analysis. As a result, we can only extract plaintext data through reused functions from other call chains, and WhisperCatcher is able to identify these regular functions. Meanwhile, only 60.95% of the plaintext data can be extracted from official encryption/encoding functions, which further highlights the limitations of ThirdEye in recovering app-encrypted data.

4.4.2 Private Data Extraction. To investigate the capability of LLMs in private data extraction, we evaluate a range of different models. Based on the benchmark dataset described in Section 4.2.2, we construct a ground truth dataset by manually annotating each traffic flow with its corresponding private data types and instances for evaluation. In terms of evaluation metrics, we choose to measure micro-averaged precision, recall and F1-Score [65] of different models. Five representative models are evaluated, including GPT-4o [38], DeepSeek-v3-0324 [19], Qwen 2.5-max [43], Gemini 2.5-flash [18], and Claude Sonnet 4 [5].

The evaluation results are presented in Table 6. The results show that all models achieve high precision, recall and F1-Score, demonstrating their effectiveness in private data extraction. Specifically, Gemini 2.5-flash achieves the highest recall of 91.10%, and highest F1-Score of 91.87%, indicating its superior performance in privacy

Table 6: Private data extraction performance of various LLMs.

Model	TP	FP	FN	Precision	Recall	F1
Qwen2.5-max [43]	1416	131	202	91.53%	87.52%	89.48%
Gemini-2.5-flash [18]	**1474**	117	**144**	92.65%	**91.10%**	**91.87%**
GPT-4o [38]	1379	94	239	93.62%	85.23%	89.23%
Deepseek-v3-0324 [19]	1384	**63**	234	**95.65%**	85.54%	90.31%
Claude-Sonnet-4 [5]	1403	75	215	94.93%	86.71%	90.63%

instance extraction. Additionally, DeepSeek-v3-0324 achieves the highest precision of 95.65%, reflecting its more reliable outputs. In order to ensure both accuracy and completeness, Gemini 2.5-flash is finally selected for private data extraction. This enhances WhisperCatcher's ability to infer private data.

Answer to RQ2. Transmission-related functions identified by WhisperCatcher are limited, with 80.95% apps containing fewer than 500 key functions, which reduces the complexity of instrumentation. In addition, all LLMs exhibit strong capabilities in private data extraction, and the Gemini 2.5-flash model is finally employed for its high precision and F1-Score, ensuring accurate and comprehensive analysis.

4.5 RQ3: Large-Scale Measurement in the Wild

After employing WhisperCatcher on the large-scale dataset with 14,879 apps, we captured 40,390 traffic flows transmitted before user consents to the privacy policy. Among them, 13,966 traffic flows were identified transmitting user's private data, including 3,838 (27.48%) flows containing app-encrypted data.

Finally, 4,966 apps are identified with UDT behaviors, Table 7 presents the transmitted privacy instances across different categories of traffic, the distribution of these instances, and their representations in traffic flows. We observe that these representations appear in various forms, such as acronyms, word prefixes, and synonymous expressions. Moreover, the values associated with these representations also exhibit significant variation, even if they represent the same privacy instance. Among the transmitted instances, 35.20% are encrypted within the app code, and this makes it difficult for existing approaches to recover. Moreover, the most frequently transmitted private data instances are OS Version, Device Model, Network Type, Resolution, and AdvertisingID, which can be combined for accurate user tracking and profiling, posing a significant threat to user privacy. For more sensitive private data such as Storage Info, MAC Address, IP Address, SSID, Location, App List and Clipboard Data, a larger number of apps choose to encrypt the data within the app itself to evade regular detection. This undoubtedly increases the potential risk of privacy leakage.

Furthermore, we explore the domains that collect user's private data. WhisperCatcher identified a total of 59,541 private data instances transmitted to 3,665 different domains. The top 10 domains are ranked based on the volume of private data they collected, as illustrated in Figure 8. We observe that the domains that most frequently collecting user's private data are typically third-party domains rather than the app's own servers. This is likely due to the integration of third-party services or the inclusion of third-party libraries or SDKs within the apps. In addition, compared to other

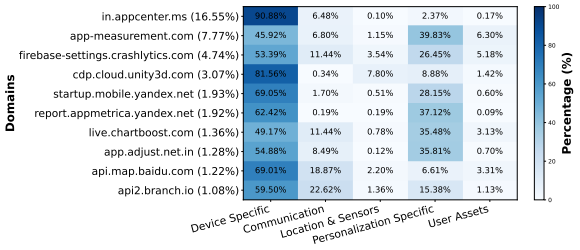


Figure 8: Top 10 domains that collect the most private data

data types, these domains collect more Device Specific and Personalization Specific private data. This may be attributed to the relative ease of accessing such data, and its effectiveness for user identification and tracking. These findings call for increased attention from both app developers and regulatory authorities.

Answer to RQ3. After conducting a large-scale measurement of 14,879 apps, WhisperCatcher identifies 4,966 apps with UDT behaviors, and 35.20% of private data instances are encrypted within the app code before transmission. The majority of private data is transmitted to third-party domains, and the most frequently transmitted data types are device specific and personalization specific.

5 Threats to Validity

Unauthorized Private Data Transmission at Startup Phase. In this work, we focus on unauthorized private data transmission behaviors at apps' startup phase, typically before the user agrees to privacy policy. However, after the user starts using the app, it may also collect and transmit private data, particularly through app-encrypted traffic. Automatically interacting with real-world apps is time-consuming, and the variation in login page designs, which are often required before accessing app features, poses significant challenges for automated bypass. So we focus on apps' traffic flows during startup phase in this study, and we take the analysis with broader interactions as future work.

Encrypted Data Transmission Implemented in Native Code. During our analysis, we find that some apps implement private data encryption and transmission in native code [56, 58, 68]. WhisperCatcher does not support native code analysis and may fail when encryption and transmission are performed at native layer. Based on rich traffic semantics, we observe that WhisperCatcher may detect reused functions in Java layer, such as data packaging functions, which allow it to recover part of private data that is actually encrypted and transmitted in native layer. This identification is inherently unstable and may impact its effectiveness. To address this limitation, WhisperCatcher can be integrated with cross-language analysis tools to enable more reliable identification of private data encrypted and transmitted in native layer.

Protection Measures of Apps. Some apps adopt advanced protection techniques, such as heavy code obfuscation [24], packers [20, 22, 62] and instrumentation detection [10, 50], to resist reverse engineering. If an app employs these protection measures,

Table 7: Unauthorized private data transmission in large-scale apps.

Data Type	Instance	Plaintext/HTTPS-Enc	App-Enc	#Apps	Selected Representations
Device Specific	IMEI	319	240	325	imei, get_imei, device_main_imei, imei_zt, imei_md5
	IMSI	27	81	60	imsi, masterImsi, RegistDeviceInfo.imsi, device.sim_slots.imsi
	UDID	77	79	93	udid, openudid, clientudid
	DeviceID	2399	690	1432	common.deviceid, device_id, device.mrgs_device_id, d_id
	Device Model	7268	2657	3670	model, meta.machine, msg.device_name, devtypid, a_na, dn, de, md, d.mo
	CPU Model	630	663	690	cpu_type, hardware, msg.cpu, cpu_abi, cpuName, cpu_platform, cpu_bran
	OemName	4481	1756	2401	deviceOEM, manufacturer, manuf, brand_name, machine, ma, mf, ua
	OS Version	7900	2483	3766	os_version, deviceOSVersion, osver, osv, osvf, os
	SIM Serial	10	10	16	device.sim_slots.iccid, sim_serial_number, masterIccid
	Resolution	3990	873	2015	screen_size, screen_width/height, x/ydpi, x/y_px, scrnh/w, dx/y, dh/w, mh/w
	SD Card Serial	8	11	19	/sys/block/mmcblk0/device/cid, sd_card_total, sd_card_free, sdCardAvailable
	Storage Info	462	886	714	msg.ram, free_memory/space, sdcard_storage, internal_storage, fm, fs, lmt
Communication	Operator	218	215	305	operator, carrier, phoneOperator, Oper
	Network Type	3179	1516	2163	net_type, network, connectivity_type, connt, grid, ntf, net, #ac
	MAC Address	363	404	407	mac, mc, wlan_mac, device.mac, header.mc, mac_addr
	IP Address	424	686	502	user.ip_address, local_ip, LoginClientIP, ipgeoinfo, conn_ip, cip, ips
	SSID	36	132	99	ssid, wss, ss, wifi_name, wifiSsid, last_probed_network_name
	BSSID	154	60	171	bssid, data.bssid, event_properties.details.bssid
	Bluetooth Info	55	90	115	bt, bluetoothMac, data.bluetooth_name, android.hardware.bluetooth_le
Location & Sensors	Location	251	391	255	coox/y, position, latitude, longitude, lat, lng, country, city, region, province
	Misc.sensors	203	389	358	deviceData.sensors, sensors.sN, sensors.sV, sensors.sVS
Personalization Specific	OAID	334	152	236	oaid,keyDeviceInfo.oaid, msaOAID, cnOAID
	UUID	2088	2120	1694	uuid, guid, rid, android_uuid
	AndroidID	1240	806	1048	aid, anid, andr_id, androidId, android_id_sha1, qg_androidId, jc_android_id
	AdvertisingID	1763	1825	1599	adid, idfa, adv_id, AdvertisingId, advertiser_id, gaid, gps_adid
	App List	257	758	615	app_list, data.pkg, pkglist, getRunningAppProcesses, currentInstalledApps
User Assets	Phone Number	73	110	121	phonenumber, phoneNo, phoneNum
	SMS	2	25	20	sms, READ_SMS, smsareacode
	Account Info	368	778	633	accountId, #account_id, getUserInfo, deviceAccount
	Clipboard Data	4	72	30	clipboard, readClipboard, pasteboard
Total Instances		38583	20958	4966	/

WhisperCatcher may fail to identify and instrument key transmission and encryption functions, and this may affect the performance. To address this issue, certain countermeasures such as de-obfuscation and unpacking, can be integrated with WhisperCatcher to enhance the identification of private data transmissions.

6 Related Work

Private data transmission analysis of mobile apps. Data transmission from app traffic is an important perspective for analyzing privacy leakages of mobile apps. Nguyen et al. [36] first conducted the large-scale measurement of Android apps to understand private data transmission behaviors without GDPR's explicit consent. To address the limitations of app traffic approaches caused by traffic mixing, proxy evasion, and SSL pinning, Cui et al. [17] proposed TraceDroid, which leverages hooking technique to capture traffic. However, TraceDroid can only handle plaintext traffic or HTTPS-encrypted traffic. To analyze private data transmitted through covert channels and encryptions, Pourali et al. [41] proposed ThirdEye, which utilizes instrumentation to recover encrypted data from official library encryptions, but it cannot handle third-party library encryptions or customized encryptions. To the best of our knowledge, WhisperCatcher is the first study that focuses on private data transmission under various app encryptions.

Privacy leakage analysis. To identify privacy leakage issues of mobile apps, researchers have proposed various approaches, including static analysis [7, 14, 37, 42, 53, 58], dynamic analysis [31, 52] and hybrid analysis [9, 63, 64, 67]. Based on these program analysis approaches, a range of studies revealed privacy issues from real-world apps. For example, Chen et al. [15] proposed ClipboardScope and revealed privacy leakage associated with clipboard usage in mobile apps. Li et al. [29] proposed XPOChecker and identified privacy leakage from user-to-user platform features of mobile apps. Continella et al. [16] proposed AGRIGENTO to detect privacy leakage of mobile apps through black-box differential analysis. Slavin et al. [48] proposed a information flow analysis framework to detect privacy policy violations of mobile apps. In addition, Bui et al. [12] proposed PurPliance to detect inconsistency from apps' traffic behaviors and privacy statements. Different from these existing studies, our work focuses on identifying unauthorized and encrypted private data transmission of mobile apps.

7 Conclusion

Android apps may encrypt and transmit user's private data without authorization, posing serious privacy risks. In this work, we propose WhisperCatcher, an automated tool for unauthorized data transmission detection. It leverages traffic semantics to guide static analysis and recover app-encrypted data through instrumentation. We conduct comprehensive evaluations and large-scale measurement analysis, and the results demonstrate that WhisperCatcher is more effective than existing approaches in identifying transmitted private data.

Acknowledgments

We sincerely thank the anonymous reviewers for their valuable feedback and insightful suggestions. This work was supported by National Natural Science Foundation of China (62232014, 62272377).

References

[1] Ahupp. 2025. python-magic. https://github.com/ahupp/python-magic

[2] Marco Alecci, Pedro Jesús Ruiz Jiménez, Kevin Allix, Tegawendé F. Bissyandé, and Jacques Klein. 2024. AndroZoo: A Retrospective with a Glimpse into the Future. In *21st IEEE/ACM International Conference on Mining Software Repositories (MSR)*. ACM, Lisbon, Portugal, 389–393.

[3] Kevin Allix, Tegawendé F. Bissyandé, Jacques Klein, and Yves Le Traon. 2016. AndroZoo: collecting millions of Android apps for the research community. In *Proceedings of the 13th International Conference on Mining Software Repositories (MSR)*. ACM, Austin, TX, USA, 468–471.

[4] Benjamin Andow, Samin Yaseer Mahmud, Justin Whitaker, William Enck, Bradley Reaves, Kapil Singh, and Serge Egelman. 2020. Actions Speak Louder than Words: Entity-Sensitive Privacy Policy and Data Flow Analysis with PoliCheck. In *29th USENIX Security Symposium (USENIX Security)*. USENIX Association, Online Event, 985–1002.

[5] Anthropic. 2025. Claude Sonnet 4: Hybrid reasoning model with superior intelligence for high-volume use cases, and 200K context window. https://www.anthropic.com/claude/sonnet

[6] APKPure. 2025. APKPure. https://apkpure.com/

[7] Steven Arzt and Eric Bodden. 2016. StubDroid: automatic inference of precise data-flow summaries for the android framework. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. ACM, Austin, TX, USA, 725–735.

[8] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick D. McDaniel. 2014. FlowDroid: precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. ACM, Edinburgh, United Kingdom, 259–269.

[9] Subarno Banerjee, David Devecsery, Peter M. Chen, and Satish Narayanasamy. 2019. Iodine: Fast Dynamic Taint Tracking Using Rollback-free Optimistic Hybrid Analysis. In *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, San Francisco, CA, USA, 490–504.

[10] Stefano Berlato and Mariano Ceccato. 2020. A large-scale study on the adoption of anti-debugging and anti-tampering protections in android apps. *J. Inf. Secur. Appl.* 52 (2020), 102463.

[11] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Annual Conference on Neural Information Processing Systems (NeurIPS)*. Curran Associates, Inc., Online Event, 1877–1901.

[12] Duc Bui, Yuan Yao, Kang G. Shin, Jong-Min Choi, and Junbum Shin. 2021. Consistency Analysis of Data-Usage Purposes in Mobile Apps. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, Virtual Event, Republic of Korea, 2824–2843.

[13] Paolo Calciati, Konstantin Kuznetsov, Alessandra Gorla, and Andreas Zeller. 2020. Automatically Granted Permissions in Android apps: An Empirical Study on their Prevalence and on the Potential Threats for Privacy. In *17th International Conference on Mining Software Repositories (MSR)*. ACM, Seoul, Republic of Korea, 29-30 June, 2020, 114–124.

[14] Yinzhi Cao, Yanick Fratantonio, Antonio Bianchi, Manuel Egele, Christopher Kruegel, Giovanni Vigna, and Yan Chen. 2015. EdgeMiner: Automatically Detecting Implicit Control Flow Transitions through the Android Framework. In *22nd Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society, San Diego, California, USA.

[15] Yongliang Chen, Ruoqin Tang, Chaoshun Zuo, Xiaokuan Zhang, Lei Xue, Xiapu Luo, and Qingchuan Zhao. 2024. Attention! Your Copied Data is Under Monitoring: A Systematic Study of Clipboard Usage in Android Apps. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*. ACM, Lisbon, Portugal, 63:1–63:13.

[16] Andrea Continella, Yanick Fratantonio, Martina Lindorfer, Alessandro Puccetti, Ali Zand, Christopher Kruegel, and Giovanni Vigna. 2017. Obfuscation-Resilient Privacy Leak Detection for Mobile Apps Through Differential Analysis. In *24th Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society, San Diego, California, USA.

[17] Huajun Cui, Guozhu Meng, Yan Zhang, Weiping Wang, Dali Zhu, Ting Su, Xiaodong Zhang, and Yuejun Li. 2022. TraceDroid: A Robust Network Traffic Analysis Framework for Privacy Leakage in Android Apps. In *Science of Cyber Security - 4th International Conference (SciSec)*. Springer, Matsue, Japan, 541–556.

[18] Google DeepMind. 2025. Gemini 2.5 Flash: Best for fast performance on everyday tasks. https://deepmind.google/models/gemini/flash/

[19] DeepSeek. 2025. DeepSeek v3: Advanced AI Language Model. https://deepseekv3.org/

[20] Zikan Dong, Hongxuan Liu, Liu Wang, Xiapu Luo, Yao Guo, Guoai Xu, Xusheng Xiao, and Haoyu Wang. 2022. What did you pack in my app? a systematic analysis of commercial Android packers. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, Singapore, Singapore, 1430–1440.

[21] DroidSSL. 2025. DroidSSLUnpinning. https://github.com/WooyunDota/DroidSSLUnpinning

[22] Yue Duan, Mu Zhang, Abhishek Vasisht Bhaskar, Heng Yin, Xiaorui Pan, Tongxin Li, Xueqiang Wang, and XiaoFeng Wang. 2018. Things You May Not Know About Android (Un)Packers: A Systematic Study based on Whole-System Emulation. In *25th Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society, San Diego, California, USA.

[23] Frida. 2025. Frida • A world-class dynamic instrumentation toolkit. https://frida.re/

[24] Cuiying Gao, Minghui Cai, Shuijun Yin, Gaozhun Huang, Heng Li, Wei Yuan, and Xiapu Luo. 2023. Obfuscation-Resilient Android Malware Analysis Based on Complementary Features. *IEEE Trans. Inf. Forensics Secur.* 18 (2023), 5056–5068.

[25] GDPR. 2025. Art. 6 GDPR Lawfulness of processing. https://gdpr.eu/article-6-how-to-process-personal-data-legally

[26] Google. 2025. Android API reference. https://developer.android.com/reference

[27] Google. 2025. Tink Cryptographic Library. https://developers.google.com/tink

[28] Yongzhong He, Xuejun Yang, Binghui Hu, and Wei Wang. 2019. Dynamic privacy leakage analysis of Android third-party libraries. *J. Inf. Secur. Appl.* 46 (2019), 259–270.

[29] Shuai Li, Zhemin Yang, Nan Hua, Peng Liu, Xiaohan Zhang, Guangliang Yang, and Min Yang. 2022. Collect Responsibly But Deliver Arbitrarily?: A Study on Cross-User Privacy Leakage in Mobile Apps. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, Los Angeles, CA, USA, 1887–1900.

[30] Yuejun Li, Huajun Cui, Jiyan Sun, Yan Zhang, Yueqi Li, Guozhu Meng, and Weiping Wang. 2022. Automated Privacy Network Traffic Detection via Self-labeling and Learning. In *IEEE Symposium on Computers and Communications (ISCC)*. IEEE, Rhodes, Greece, 1–8.

[31] Björn Mathis, Vitalii Avdiienko, Ezekiel O. Soremekun, Marcel Böhme, and Andreas Zeller. 2017. Detecting information flow by mutating input data. In *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE Computer Society, Urbana, IL, USA, 263–273.

[32] Mark Huasong Meng, Chuan Yan, Yun Hao, Qing Zhang, Zeyu Wang, Kailong Wang, Sin Gee Teo, Guangdong Bai, and Jin Song Dong. 2024. A Large-Scale Privacy Assessment of Android Third-Party SDKs. *CoRR* abs/2409.10411 (2024). arXiv:2409.10411

[33] Mark Huasong Meng, Qing Zhang, Guangshuai Xia, Yuwei Zheng, Yanjun Zhang, Guangdong Bai, Zhi Liu, Sin G. Teo, and Jin Song Dong. 2023. Post-GDPR Threat Hunting on Android Phones: Dissecting OS-level Safeguards of User-unresettable Identifiers. In *30th Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society, San Diego, California, USA.

[34] Mitmproxy. 2025. Mitmproxy - an interactive HTTPS proxy. https://mitmproxy.org/

[35] Yuhong Nan, Zhemin Yang, Xiaofeng Wang, Yuan Zhang, Donglai Zhu, and Min Yang. 2018. Finding Clues for Your Secrets: Semantics-Driven, Learning-Based Privacy Discovery in Mobile Apps. In *25th Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society, San Diego, California, USA.

[36] Trung Tin Nguyen, Michael Backes, Ninja Marnau, and Ben Stock. 2021. Share First, Ask Later (or Never?) Studying Violations of GDPR's Explicit Consent in Android Apps. In *30th USENIX Security Symposium (USENIX Security)*. USENIX Association, Online Event, 3667–3684.

[37] Damien Octeau, Daniel Luchaup, Matthew L. Dering, Somesh Jha, and Patrick D. McDaniel. 2015. Composite Constant Propagation: Application to Android Inter-Component Communication Analysis. In *37th IEEE/ACM International Conference on Software Engineering (ICSE)*. IEEE Computer Society, Florence, Italy, 77–88.

[38] OpenAI. 2025. Hello GPT-4o. https://openai.com/index/hello-gpt-4o/

[39] Openatx. 2025. Adbutils. https://github.com/openatx/adbutils

[40] PIPL. 2025. Personal Information Protection Law of the People's Republic of China. http://en.npc.gov.cn.cdurl.cn/2021-12/29/c_694559.htm

[41] Sajjad Pourali, Nayanamana Samarasinghe, and Mohammad Mannan. 2022. Hidden in Plain Sight: Exploring Encrypted Channels in Android Apps. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, Los Angeles, CA, USA, 2445–2458.

[42] Lina Qiu, Yingying Wang, and Julia Rubin. 2018. Analyzing the analyzers: FlowDroid/IccTA, AmanDroid, and DroidSafe. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, Amsterdam, The Netherlands, 176–186.

[43] Qwenlm. 2025. Qwen2.5-Max: Exploring the Intelligence of Large-scale MoE Model. https://qwenlm.github.io/blog/qwen2.5-max/

[44] Joel Reardon, Álvaro Feal, Primal Wijesekera, Amit Elazari Bar On, Narseo Vallina-Rodriguez, and Serge Egelman. 2019. 50 Ways to Leak Your Data: An Exploration of Apps' Circumvention of the Android Permissions System. In *28th USENIX*

Security Symposium (USENIX Security). USENIX Association, Santa Clara, CA, USA, 603–620.

[45] Cristiana Teixeira Santos, Arianna Rossi, Lorena Sánchez Chamorro, Kerstin Bongard-Blanchy, and Ruba Abu-Salma. 2021. Cookie Banners, What's the Purpose?: Analyzing Cookie Banner Text Through a Legal Lens. In *Proceedings of the 20th Workshop on Workshop on Privacy in the Electronic Society (WPES)*. ACM, Virtual Event, Korea, 187–194.

[46] Theodor Schnitzler, Katharina Kohls, Evangelos Bitsikas, and Christina Pöpper. 2023. Hope of Delivery: Extracting User Locations From Mobile Instant Messengers. In *30th Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society, San Diego, California, USA.

[47] Skylot. 2025. jadx - Dex to Java decompiler. https://github.com/skylot/jadx

[48] Rocky Slavin, Xiaoyin Wang, Mitra Bokaei Hosseini, James Hester, Ram Krishnan, Jaspreet Bhatia, Travis D. Breaux, and Jianwei Niu. 2016. Toward a framework for detecting privacy policy violations in android application code. In *Proceedings of the 38th International Conference on Software Engineering (ICSE)*. ACM, Austin, TX, USA, 25–36.

[49] Soot. 2025. Soot-oss/soot - A Java optimization framework. https://github.com/soot-oss/soot

[50] Enrique Soriano-Salvador and Gorka Guardiola Muzquiz. 2023. Detecting and bypassing frida dynamic function call tracing: exploitation and mitigation. *J. Comput. Virol. Hacking Tech.* 19, 4 (2023), 503–513.

[51] Square. 2025. OkHttp. https://square.github.io/okhttp/

[52] Mingshen Sun, Tao Wei, and John C. S. Lui. 2016. TaintART: A Practical Multi-level Information-Flow Tracking System for Android RunTime. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, Vienna, Austria, 331–342.

[53] Xiaoyu Sun, Xiao Chen, Li Li, Haipeng Cai, John Grundy, Jordan Samhi, Tegawendé F. Bissyandé, and Jacques Klein. 2023. Demystifying Hidden Sensitive Operations in Android Apps. *ACM Trans. Softw. Eng. Methodol.* 32, 2 (2023), 50:1–50:30.

[54] Junjie Tao, Jifei Shi, Ming Fan, Yin Wang, Junfeng Liu, and Ting Liu. 2023. JSLibD: Reliable and Heuristic Detection of Third-party Libraries in Miniapps. In *Proceedings of the 2023 ACM Workshop on Secure and Trustworthy Superapps (SaTS)*. ACM, Copenhagen, Denmark, 11–16.

[55] Wandoujia. 2025. Wandoujia. https://www.wandoujia.com/

[56] Jikai Wang and Haoyu Wang. 2024. NativeSummary: Summarizing Native Binary Code for Inter-language Static Analysis of Android Apps. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, Vienna, Austria, 971–982.

[57] Yin Wang, Ming Fan, Junfeng Liu, Junjie Tao, Wuxia Jin, Haijun Wang, Qi Xiong, and Ting Liu. 2024. Do as You Say: Consistency Detection of Data Practice in Program Code and Privacy Policy in Mini-App. *IEEE Trans. Software Eng.* 50, 12 (2024), 3225–3248.

[58] Fengguo Wei, Sankardas Roy, Xinming Ou, and Robby. 2018. Amandroid: A Precise and General Inter-component Data Flow Analysis Framework for Security Vetting of Android Apps. *ACM Trans. Priv. Secur.* 21, 3 (2018), 14:1–14:32.

[59] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Annual Conference on Neural Information Processing Systems (NeurIPS)*. Curran Associates, Inc., New Orleans, LA, USA, 24824–24837.

[60] Jiangrong Wu, Yuhong Nan, Luyi Xing, Jiatao Cheng, Zimin Lin, Zibin Zheng, and Min Yang. 2024. Leaking the Privacy of Groups and More: Understanding Privacy Risks of Cross-App Content Sharing in Mobile Ecosystem. In *31st Annual Network and Distributed System Security Symposium (NDSS)*. The Internet Society, San Diego, California, USA.

[61] Zhiwu Xu, Xiongya Hu, Yida Tao, and Shengchao Qin. 2020. Analyzing Cryptographic API Usages for Android Applications Using HMM and N-Gram. In *International Symposium on Theoretical Aspects of Software Engineering (TASE)*. IEEE, Hangzhou, China, 153–160.

[62] Lei Xue, Hao Zhou, Xiapu Luo, Le Yu, Dinghao Wu, Yajin Zhou, and Xiaobo Ma. 2022. PackerGrind: An Adaptive Unpacking System for Android Apps. *IEEE Trans. Software Eng.* 48, 2 (2022), 551–570.

[63] Yufei Yang, Wenbo Luo, Yu Pei, Minxue Pan, and Tian Zhang. 2019. Execution Enhanced Static Detection of Android Privacy Leakage Hidden by Dynamic Class Loading. In *43rd IEEE Annual Computer Software and Applications Conference (COMPSAC)*. IEEE, Milwaukee, WI, USA, 149–158.

[64] Zhi Yang, Zhanhui Yuan, Shuyuan Jin, Xingyuan Chen, Lei Sun, Xuehui Du, Wenfa Li, and Hongqi Zhang. 2022. FSAFlow: Lightweight and Fast Dynamic Path Tracking and Control for Privacy Protection on Android Using Hybrid Analysis with State-Reduction Strategy. In *43rd IEEE Symposium on Security and Privacy (SP)*. IEEE, San Francisco, CA, USA, 2114–2129.

[65] Analytics Yogi. 2023. Micro-average, Macro-average, Weighting: Precision, Recall, F1-Score. https://vitalflux.com/micro-average-macro-average-scoring-metrics-multi-class-classification-python/

[66] Lei Zhang, Zhibo Zhang, Ancong Liu, Yinzhi Cao, Xiaohan Zhang, Yanjun Chen, Yuan Zhang, Guangliang Yang, and Min Yang. 2022. Identity Confusion in WebView-based Mobile App-in-app Ecosystems. In *31st USENIX Security Symposium (USENIX Security)*. USENIX Association, Boston, MA, USA, 1597–1613.

[67] Mu Zhang and Heng Yin. 2014. Efficient, context-aware privacy leakage confinement for android applications without firmware modding. In *9th ACM Symposium on Information, Computer and Communications Security (CCS)*. ACM, Kyoto, Japan, 259–270.

[68] Hao Zhou, Shuohan Wu, Xiapu Luo, Ting Wang, Yajin Zhou, Chao Zhang, and Haipeng Cai. 2022. NCScope: hardware-assisted analyzer for native code in Android apps. In *31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, Virtual Event, South Korea, 629–641.

[69] Lu Zhou, Chengyongxiao Wei, Tong Zhu, Guoxing Chen, Xiaokuan Zhang, Suguo Du, Hui Cao, and Haojin Zhu. 2023. POLICYCOMP: Counterpart Comparison of Privacy Policies Uncovers Overbroad Personal Data Collection Practices. In *32nd USENIX Security Symposium (USENIX Security)*. USENIX Association, Anaheim, CA, USA, 1073–1090.